

SingleCell CLI Pipeline - Project Context

Vision

The goal of this project is to develop a modular, reproducible, and configurable command-line pipeline for single-cell RNA-seq analysis.

The pipeline should allow users with varying computational resources and dataset sizes to run a standardized workflow while retaining the ability to customize analysis parameters through configuration files.

The project is not intended to be another analysis script. It should function as a software package that can be installed, configured, and executed from the command line.

Version 1 will use a fixed set of analysis tools (primarily Scanpy-based methods), while future versions may allow users to choose between multiple tools for each analysis step.

Design Principles

Modular

Each analysis step should be implemented independently and be replaceable in the future.

Config-Driven

Users should configure the pipeline through YAML files rather than modifying source code.

Reproducible

Running the same command with the same configuration should produce identical results.

Extensible

Future methods can be added without changing the CLI interface.

Resource-Aware

Users should be able to run the pipeline on laptops, workstations, or large servers using different presets.

Scope (Version 1)

Input:

- h5ad

- 10x h5
- 10x mtx

Pipeline: 1. Data Loading 2. Quality Control 3. Cell/Gene Filtering 4. Normalization 5. Highly Variable Gene Selection 6. Scaling 7. PCA 8. Neighbor Graph Construction 9. UMAP 10. Clustering 11. Marker Detection

Output:

- Processed AnnData object
- QC reports
- Cluster assignments
- Marker gene tables
- Publication-ready figures

Cell-type annotation and batch integration may be added in future releases.

Project Structure

singlecell-pipeline/

```

├── pyproject.toml ├── README.md ├── LICENSE ├── .gitignore | ├── configs/ | ├──
default.yaml | ├── laptop.yaml | ├── workstation.yaml | ├── large.yaml | ├── docs/ |
├── context.md | ├── architecture.md | ├── cli.md | ├── modules.md | ├── tests/ |
├── test_io.py | ├── test_qc.py | ├── test_config.py | ├── example_data/ | ├──
pbmc3k.h5ad | ├── singlecell/ | | | ├── init.py | | | ├── cli/ | | | ├── init.py | |
├── main.py | | | ├── core/ | | | ├── config.py | | | ├── logger.py | | | ├── pipeline.py
| | | ├── exceptions.py | | | ├── io/ | | | ├── reader.py | | | ├── writer.py | | | ├──
modules/ | | | ├── qc.py | | | ├── filtering.py | | | ├── normalization.py | | | ├── hvg.py |
| | | ├── scaling.py | | | ├── pca.py | | | ├── neighbors.py | | | ├── clustering.py | | | ├──
markers.py | | | ├── annotation.py | | | ├── plots/ | | | ├── qc_plots.py | | | ├──
umap.py | | | ├── markers.py | | | ├── reports/ | | | ├── report_generator.py | | | ├──
utils/ | | | ├── paths.py | | | ├── helpers.py | | | ├── outputs/

```

File Responsibilities

Root Files

pyproject.toml

Project metadata, dependencies, and installation configuration.

README.md

User-facing documentation and installation guide.

LICENSE

Defines project licensing terms.

.gitignore

Specifies files and directories ignored by Git.

Configurations

configs/default.yaml

Default pipeline parameters.

configs/laptop.yaml

Preset for low-resource systems.

configs/workstation.yaml

Preset for medium-to-high resource systems.

configs/large.yaml

Preset for large datasets and servers.

Documentation

docs/context.md

Project vision, scope, and development decisions.

docs/architecture.md

Technical architecture and execution flow.

docs/cli.md

CLI commands and argument reference.

docs/modules.md

Documentation of analysis modules.

Testing

tests/test_io.py

Tests for reading and writing datasets.

tests/test_qc.py

Tests for QC calculations and filtering logic.

tests/test_config.py

Tests for configuration parsing and validation.

Example Data

example_data/pbmc3k.h5ad

Example dataset for development and testing.

Package

singlecell/init.py

Marks the package and exposes package metadata.

CLI

singlecell/cli/init.py

Marks CLI package.

singlecell/cli/main.py

Entry point for all command-line operations.

Core Infrastructure

singlecell/core/config.py

Loads and validates configuration files.

singlecell/core/logger.py

Provides centralized logging.

singlecell/core/pipeline.py

Controls execution order of analysis modules.

singlecell/core/exceptions.py

Defines custom exceptions and error handling.

Input / Output

singlecell/io/reader.py

Loads supported data formats into AnnData.

singlecell/io/writer.py

Saves processed datasets and results.

Analysis Modules

singlecell/modules/qc.py

Computes quality control metrics.

singlecell/modules/filtering.py

Filters low-quality cells and genes.

singlecell/modules/normalization.py

Normalizes count matrices.

singlecell/modules/hvg.py

Selects highly variable genes.

singlecell/modules/scaling.py

Scales expression values.

singlecell/modules/pca.py

Performs dimensionality reduction using PCA.

singlecell/modules/neighbors.py

Builds nearest-neighbor graphs.

singlecell/modules/clustering.py

Performs Leiden/Louvain clustering.

singlecell/modules/markers.py

Identifies cluster marker genes.

singlecell/modules/annotation.py

Assigns cell-type labels.

Visualization

singlecell/plots/qc_plots.py

Generates QC-related figures.

singlecell/plots/umap.py

Generates UMAP visualizations.

singlecell/plots/markers.py

Generates marker-gene visualizations.

Reporting

singlecell/reports/report_generator.py

Creates summary reports and result documents.

Utilities

singlecell/utis/paths.py

Manages project and output paths.

singlecell/utis/helpers.py

Contains reusable helper functions.

Outputs

outputs/

Stores all generated results, figures, logs, and processed datasets.

Immediate Development Goal

Build a minimal working pipeline capable of:

1. Loading an h5ad file.
2. Reading a YAML configuration.
3. Running QC.
4. Producing QC metrics.
5. Saving outputs.

Only after this foundation is stable should additional modules be added.

The objective is not to build every feature immediately, but to create an architecture where new modules can be added without redesigning the project.